

# PROMPTVER: Semantic Versioning for Prompt Edits

Author Name(s) Omitted for Review

Institution Omitted for Review

Email Omitted for Review

**Abstract**—Large language model (LLM)-based applications evolve through prompt edits the way traditional software evolves through code edits, but without an equivalent versioning discipline. Code semantic versioning does not transfer directly: prompts are natural-language artifacts whose semantics emerge at execution time, and the relevant contract surface is implicit in the joint behavior of prompt, model, input schema, and downstream code.

We present PROMPTVER, a semantic versioning framework that adapts SemVer’s PATCH/MINOR/MAJOR discipline to prompt edits. PROMPTVER treats a deployable prompt artifact as a configuration anchored on the prompt template  $P$  and pinned model snapshot  $M$ , decomposes  $P$  into role, directive, contextual constraints, examples, and output schema, defines a 20-operator edit taxonomy, and applies a rule-guided procedure that assigns a SemVer label from the prompt-pair diff alone. Reference labels are derived from execution against a paraphrastic noise floor and augmented with hard-violation checks for contract-level failures.

On 116 controlled edit-clusters across classification and summarization tasks (436 valid edit-executor rows), PROMPTVER reaches 0.718 exact agreement with the empirical reference, against an always-PATCH floor of 0.670 (macro-F1 0.267, dangerous-miss rate 1.000). Accuracy is stronger on classification (0.730) than on XSum summarization (0.684), where a direct LLM baseline using only SemVer definitions remains competitive. The result is therefore not a claim that rule-guidance alone solves prompt versioning. Rather, PROMPTVER’s main value is a reproducible decision trace : affected component, contract checks, triggering rule, and structured justification. That turns a black-box severity prediction into an auditable release decision. We report preliminary evidence on classification and summarization; broader transfer remains future work.

**Index Terms**—prompt engineering, semantic versioning, LLM applications, prompt evolution, software maintenance.

## I. INTRODUCTION

In May 2024, a maintainer of `dataherald`, a natural-language-to-SQL platform, committed a one-line “fix” to a SQL-evaluation prompt. The previous template ended with “give me a one or two lines explanation and the score after `Score:` ”. After the edit, the score directive had been dropped and a new template variable `{SQL_result}` had been added. Textually the diff was minimal; semantically the contract had broken in two places: callers had not been updated to provide `{SQL_result}` (causing runtime exceptions), and downstream parsers depending on the literal `Score:`  prefix silently broke. There was no version bump, no changelog, no migration guidance.

Anecdotes like this are increasingly common as LLM-based applications proliferate [21], [22]. Mining studies of prompt-edit histories report thousands of similar commits across production repositories: adding constraints, changing output schemas, swapping role attributions, modifying example sets, most receiving no version-management treatment [6], [21]. Practitioners describe ad-hoc workarounds : copying prompts into spreadsheets, manually re-running regression tests, or accepting silent regressions [17]. The problem is precisely the one that semantic versioning (SemVer) was designed to solve for code [3], [12].

Yet code-SemVer’s PATCH/MINOR/MAJOR discipline does not transfer directly to prompt edits. Three properties make direct application impossible. First, prompts are natural-language artifacts with no abstract syntax tree, no API surface, and no type system; textual diffs and edit-distance heuristics conflate cosmetic rephrasing with behavior-altering substitution [4], [15]. Second, prompt semantics emerge only at execution: the same edit can produce different behavioral severity depending on the executor model, input distribution, and downstream parsing logic [5], [19]. Third, prompts have no stable contract: a function exposes its signature, but a prompt’s contract is implicit in the joint behavior of prompt, model, input schema, and downstream code, any of which can drift [18].

We present PROMPTVER, a semantic versioning framework that adapts SemVer to prompt edits. PROMPTVER rests on four components: (i) a *versioning unit*  $U = \{P, H, M, C, T\}$  capturing the configuration a downstream consumer depends on, with the template  $P$  further decomposed into  $\langle R, D, Ctx, E, O \rangle$ ; (ii) a 20-operator edit taxonomy grounded in prior empirical work on instruction, constraint, example, format, role, and surface edits; (iii) a rule-guided classification procedure that assigns a SemVer label from the prompt-pair diff alone, without re-execution at deployment; and (iv) behavioral labeling rules that derive an empirical reference from execution against a paraphrastic noise floor, augmented with hard contract-level checks. The framework provides a deployment-time severity estimate from prompt structure, with execution-derived labels used only for validation. It is not a behavioral oracle: it is a structured, auditable mechanism for communicating likely release impact.

**Contributions.** (1) PROMPTVER, a semantic versioning framework for prompt edits; (2) PROMPTVER-BENCH, a

controlled benchmark with execution-derived labels calibrated against a paraphrastic noise floor and hard-violation checks; (3) a preliminary empirical evaluation across 436 valid edit-executor rows, showing where structured prompt diffs agree with execution-derived labels and where open-ended generation remains difficult; and (4) a comparison against a strong direct-LLM baseline showing that simple prompting is competitive, while PROMPTVER adds the structured decision trace needed for reproducible versioning decisions.

## II. THE PROMPTVER FRAMEWORK

PROMPTVER produces a SemVer label  $\ell \in \{\text{PATCH}, \text{MINOR}, \text{MAJOR}\}$  with a structured justification, defined from the perspective of downstream consumers. A MAJOR edit may break consumers; a MINOR edit extends or steers behavior without removing existing capabilities; a PATCH edit preserves intended behavior and downstream contract.

### A. Versioning Unit and Template Decomposition

A deployable prompt artifact is treated as a configuration  $U = \{P, H, M, C, T\}$ , where  $P$  is the raw template;  $H$  is the hyperparameter set (temperature, top- $k$ , seed);  $M$  is the model snapshot (a pinned identifier such as `gpt-4-0613`, never `latest`);  $C$  is the context schema (input structure or RAG contract); and  $T$  is the validation test set. *In this paper our experiments vary only  $P$  while pinning  $M$ ;  $H$ ,  $C$ , and  $T$  are fixed by the benchmark configuration.* We retain them in the framework definition because production prompt artifacts are versioned jointly with these elements (e.g., a temperature change with no  $P$  edit can still alter consumer-visible behavior), but their empirical evaluation is left to future work.

The template  $P$  is further decomposed:

$$P = \langle R, D, Ctx, E, O \rangle, \quad (1)$$

where  $R$  is an optional role/persona;  $D$  is the core directive;  $Ctx$  is a list of contextual constraint sentences;  $E$  is a list of demonstration pairs (empty for zero-shot); and  $O$  is the output-schema specification. This decomposition exposes the *contract surface*:  $D$  and  $O$  define what the prompt promises to its caller, while  $R$ ,  $Ctx$ , and  $E$  define how the model is steered. SemVer rules distinguish edits that alter the contract surface (typically MAJOR) from edits that adjust steering (typically MINOR or PATCH).

The decomposition synthesizes prior work. Tafreshpour et al. [21] empirically identified recurring components in 1,262 prompt commits across 243 repositories. Morales et al.’s Impromptu [8] formalized prompts as compositions of traits and cores. Perrone and Romano’s PEaC [11] treated prompts as YAML-serialized objects with explicit instruction, context, input, and output fields. Ronanki et al. [16] catalogued prompt smells from conflated components.

TABLE I  
PROMPTVER OPERATOR FAMILIES. THE FAMILY IDENTIFIES THE CHANGED COMPONENT; THE CLASSIFICATION PROCEDURE, NOT THE FAMILY ALONE, DETERMINES SEVERITY.

| Family      | IDs               | Comp.  | Typical transformations                                    |
|-------------|-------------------|--------|------------------------------------------------------------|
| Instruction | $E_1-E_3, E_{15}$ | $D$    | Add, remove, rewrite, or restructure task directives.      |
| Constraint  | $E_4-E_6$         | $Ctx$  | Add, remove, or modify behavioral or validity constraints. |
| Example     | $E_7-E_9, E_{16}$ | $E$    | Add, remove, modify, or reorder demonstrations.            |
| Format      | $E_{10}-E_{11}$   | $O$    | Add or change output format, fields, labels, or schema.    |
| Role        | $E_{12}-E_{14}$   | $R$    | Add, remove, or modify persona/role framing.               |
| Surface     | $E_{17}-E_{20}$   | layout | Reformat, reorder, clarify wording, or correct typos.      |

### B. Operator Taxonomy

PROMPTVER defines 20 fine-grained edit operators  $E_1-E_{20}$ , each describing a single transformation between  $P$  and  $P'$  that touches one primary component. Table I summarizes the complete operator families; the replication package gives the full operator-level catalog and examples.

The families are grounded in prior empirical work: Instruction edits ( $E_1-E_3, E_{15}$ ) influence task prioritization [1], [5]; Constraint edits ( $E_4-E_6$ ) alter the feasible response space [19]; Example edits ( $E_7-E_9, E_{16}$ ) affect in-context generalization [4]; Format edits ( $E_{10}-E_{11}$ ) affect decoding stability [19]; Role edits ( $E_{12}-E_{14}$ ) induce global behavioral shifts with high variance [8], [19]; Surface/restructuring edits ( $E_{15}-E_{20}$ ) are form-level modifications consistent with dominant patterns in real workflows [6], [15]. Beyond our controlled study, an empirical analysis of prompt-evolution histories in open-source repositories confirms that practitioners evolve prompts predominantly through incremental additions, modifications, restructurings, and corrections; the same operations this taxonomy names [21]. The taxonomy is operator-centric: family does not predetermine label. A constraint addition ( $E_4$ ) can be PATCH if it tightens cosmetic phrasing or MAJOR if it breaks a relied-upon behavior; the rule-guided procedure determines severity based on the components affected.

### C. Rule-Guided Classification Procedure

Given  $(P, P')$  and the executor model identifier, PROMPTVER produces a label via sequential structural checks (Algorithm 1). Each step inspects a specific property and may commit to a label, escalate a candidate, or defer.

**Implementation as a structured single-judge.** The procedure is realized as a single LLM call whose system prompt encodes the decision rules and whose output is structured JSON containing the neutral diff, affected components, contract checks, committed label, and step-citing justification. Three independent runs with majority voting form the deployed classifier. The judge receives the prompt pair and the executor identifier, but never observes executor outputs; its decision is structural, not behavioral. Operationally, a prompt-touching commit flows through five artifacts: prompt pair, component diff, operator assignment, contract checks, and a SemVer label with an audit record.

---

**Algorithm 1** PROMPTVER Classification Procedure

---

**Require:** prompt pair  $(P, P')$ , executor model  $M$

**Ensure:** SemVer label  $\ell \in \{\text{PATCH}, \text{MINOR}, \text{MAJOR}\}$ , triggering step

```
1: produce neutral diff over  $\langle R, D, Ctx, E, O \rangle$ 
2: identify task families  $t, t'$  from allowed set
3: if  $t \neq t'$  and the change is more than wording clarification then
4:   return (MAJOR, "task change")
5: end if
6: if required template variable or input contract changes then
7:   return (MAJOR, "input contract")
8: end if
9: if required downstream-visible output contract changes then
10:  return (MAJOR, "output contract")
11: end if
12: classify  $\Delta_{Ctx}$  as Style / Safety / Decision-rule / Hard-validity
13: if Decision-rule changes valid output distribution or Hard-
    validity changes parser assumptions then
14:  return (MAJOR, "constraint contract")
15: else if Style or Safety only then
16:   $cand \leftarrow \text{MINOR}$  (PATCH if redundant)
17: end if
18: if examples demonstrate different task or contradict directive
    then
19:  return (MAJOR, "example drift")
20: else if clean examples added/removed then
21:   $cand \leftarrow \max(cand, \text{MINOR})$  for classification; PATCH for
    clear summarization
22: end if
23: handle role/persona and surface edits
24: apply generation-specific calibration; verify final consistency
25: return ( $cand$ , triggering step)
```

---

### III. METHODOLOGY

We evaluate PROMPTVER in a controlled benchmark with behaviorally derived reference labels. The edit catalog, judge prompts, execution outputs, empirical-label scripts, and analysis code are released as a replication package.

#### A. Edit Catalog and Stratification

The catalog contains 141 prompt-edit pairs across three task families: TREC (6-way topic classification, 48 edits), DBPedia (14-way entity classification, 48 edits), and XSum (single-sentence summarization, 45 edits). For evaluation, edits are stratified into seven *evaluation groups* probing different operator clusters: **A** surface and exemplar density, **B** constraint addition/removal, **C** persona, **D** format/schema, **E** example/role combinations, **F** cross-component edits, and **G** adversarial consistency-check edits. This stratification is an evaluation device, not part of the framework definition.

#### B. Executor Models

We run every edit under four executor models: gemini-2.5-pro, gemini-2.5-flash, gemini-2.5-flash-lite, and llama-3.1-8b-instant. The judge is a separate model (gpt-4o) not in the executor set. All runs use temperature 0, top- $p = 1$ , max 512 tokens for classification and 256 for summarization. Each edit contributes one row per available executor; analyses cluster at the edit level.

#### C. Behavioral Labels via a Paraphrastic Noise Floor

A change in model output between  $P$  and  $P'$  does not automatically mean meaningful behavioral change: LLMs exhibit baseline prompt sensitivity even under semantically equivalent rephrasings [4], [19]. We calibrate a *noise floor* per task-executor pair:  $k = 30$  semantically equivalent variants of  $P_0$  are run on every input, the  $\binom{30}{2} = 435$  pairwise output distances are computed, and the 95th percentile is taken as the per-cell noise ceiling. Inputs with unstable distance distributions (median above  $\theta = 0.10$ ) are dropped, leaving the *stable witness set*  $\mathcal{D}_s$ .

For classification, we compute the flip rate on  $\mathcal{D}_s$ ; for summarization, median semantic drift using  $1 - \text{BERTScore}_{F_1}$ . Continuous metrics cannot detect all SemVer-relevant failures: a plain-text-to-JSON change preserves semantic content while breaking downstream parsers. We therefore apply hard-violation checks before metric thresholding: if the edit changes the task, input/template variables, required output schema, parser-visible structure, output-language contract, required fields, or label space, the empirical label is set to MAJOR regardless of distance. Otherwise, PATCH/MINOR/MAJOR is assigned via the calibrated noise-floor thresholds. This hard-violation-aware label is our primary empirical reference.

#### D. Baselines

**Always-PATCH.** Predicts PATCH on every input; tests whether accuracy is inflated by class distribution and quantifies the dangerous-miss cost of a maximally optimistic predictor.

**Simple-LLM SemVer judge.** Uses the same judge model and decoding protocol as PROMPTVER, but receives only the SemVer class definitions and three short examples : no rule-guided procedure, component decomposition, contract checks, or generation calibration. Tests whether a strong LLM can infer prompt-edit severity directly from the prompt pair.

Both LLM-based systems see the same prompt pairs and executor identifiers and are evaluated against the same empirical reference.

#### E. Metrics

Exact accuracy, macro-F1, Kendall's  $\tau_b$ , dangerous-miss rate  $P(\hat{\ell}=\text{PATCH} \mid \ell_{\text{emp}}=\text{MAJOR})$ , over-conservative rate  $P(\hat{\ell}=\text{MAJOR} \mid \ell_{\text{emp}}=\text{PATCH})$ , and per-class precision/recall/F1. Paired comparisons use the subset of rows joinable across both LLM systems and the empirical reference.

## IV. RESULTS

#### A. Headline Results and Trivial Baselines

After execution checks, noise-floor calibration, stable-witness filtering, and hard-violation-aware labeling, the retained benchmark contains 436 valid edit-executor rows. Table II reports accuracy and the safety diagnostics most relevant to versioning. Dangerous misses are especially important because they measure breaking edits predicted as PATCH.

The always-PATCH predictor reaches 0.670 accuracy purely because the PATCH class is the majority, but its macro-F1 collapses to 0.267 and its dangerous-miss rate is 1.000—it

TABLE II  
CONTROLLED-BENCHMARK RESULTS. ACCURACY ALONE IS INSUFFICIENT FOR VERSIONING; DANGEROUS MISSES MEASURE BREAKING EDITS PREDICTED AS PATCH.

| Method / Scope             | Acc.         | M-F1  | Dang.-miss   | Rows |
|----------------------------|--------------|-------|--------------|------|
| Always-PATCH (global)      | 0.670        | 0.267 | <b>1.000</b> | 436  |
| Simple-LLM (global)        | 0.673        | 0.586 | 0.0485       | 436  |
| PROMPTVER – Global         | <b>0.718</b> | 0.549 | 0.0483       | 436  |
| PROMPTVER – Classification | 0.730        | 0.606 | 0.0667       | 322  |
| PROMPTVER – XSum           | 0.684        | 0.525 | 0.000        | 114  |

TABLE III  
ACCURACY COMPARISON WITH THE SIMPLE-LLM SEMVER BASELINE ON MATCHED ROWS.

| Method                  | Global       | Class.       | XSum         | Audit trace |
|-------------------------|--------------|--------------|--------------|-------------|
| Always-PATCH            | 0.670        | 0.659        | 0.702        | No          |
| Simple-LLM SemVer judge | 0.673        | 0.656        | <b>0.719</b> | No          |
| PROMPTVER               | <b>0.718</b> | <b>0.730</b> | 0.684        | Yes         |

labels *every* breaking edit as a non-event. This is the worst-case operational profile: silent regressions on every MAJOR edit, including the contract failures hard-violation labels were designed to catch. PROMPTVER’s 4.8-point accuracy gain over this floor is therefore preliminary rather than decisive; the more important question is whether the system reduces dangerous misses while preserving an auditable safe-change signal.

### B. Comparison with a Simple LLM Baseline

Table III compares PROMPTVER with the Simple-LLM SemVer baseline on the matched subset.

The Simple-LLM baseline is competitive, especially on XSum where it outperforms PROMPTVER. This is an important negative result: much of the SemVer signal is already accessible to a strong LLM from the prompt pair alone. We therefore do not claim that the rule-guided procedure is primarily valuable as a higher-accuracy classifier. Its added value is that it produces an *auditable decision trace*: affected component, task-family decision, template-variable check, output-contract check, triggering rule, and structured justification. This trace makes the classifier usable as a versioning mechanism rather than only a black-box labeler.

**Qualitative trace evidence.** Table IV illustrates why the trace matters operationally. The examples are not presented as statistical proof; they show how structured fields expose the source of a decision and make both correct classifications and mistakes reviewable.

### C. Per-Task and Cross-Executor Observations

Classification is easier because the reference is categorical: label flips, label-space changes, and parse failures are directly observable. XSum is the hard case: clean example edits or style constraints may leave outputs semantically similar, while parser-visible format changes may be breaking even when BERTScore remains high. The Simple-LLM baseline’s stronger XSum result indicates that the current rule-guided procedure is over-constrained for open-ended generation. Per-group errors concentrate where structural and behavioral severity diverge (example/style and cross-component edits in XSum); format/schema changes are handled reliably once

TABLE IV  
REPRESENTATIVE TRACE CASES. PROMPTVER EXPOSES THE COMPONENT AND RULE BEHIND THE LABEL, WHILE THE DIRECT BASELINE RETURNS ONLY AN UNSTRUCTURED RATIONALE.

| Case              | PROMPTVER trace                                                     | Why it matters                                                          |
|-------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------|
| TREC $E_{10}$     | $\Delta O$ ; output-contract step; affected key emitted.            | Maintainer can identify parser risk, not just “format changed.”         |
| XSum $E_{12}+E_4$ | Constraint branch fired although reference was MINOR.               | Error becomes diagnosable: style-only constraint was over-escalated.    |
| DBPedia $E_{19}$  | Input-contract step; new template variable not supplied by callers. | Catches the placeholder-substitution failure mode missed as rephrasing. |

hard-violation labels are applied. The judge receives the executor identifier  $M$ , but the predicted label is dominated by the prompt-pair diff: in XSum, the three Gemini executors yield identical per-group profiles. We treat this as consistency, not executor-agnosticism; an  $M$ -ablated re-run is future work.

## V. DISCUSSION AND IMPLICATIONS

**Findings.** The results provide preliminary evidence that prompt-edit severity can be approximated from structured prompt diffs, but they do not show that prompt SemVer is solved. PROMPTVER reaches 0.718 global agreement, with stronger classification (0.730) than summarization (0.684) performance and a 4.8-point lift over the always-PATCH floor. A direct LLM baseline is competitive and is stronger on XSum, so PROMPTVER’s contribution is not a large accuracy gain; its value is audibility: the qualitative cases show that the trace enables root-cause analysis even on disagreements. Open-ended generation requires different calibration from classification.

**Refinements and implications.** Future work should test a *compact* rule-guided prompt that preserves audit fields with less over-constraint, make generation-specific calibration explicit rather than treating MINOR as residual, and apply hard-violation checks before metric thresholding throughout the labeling pipeline. PROMPTVER can run in continuous integration pipelines (CI) on every prompt-touching commit, surfacing PATCH/MAJOR verdicts in release notes and MINOR verdicts as “review needed.” It is not a behavioral oracle, but it gives teams a reproducible way to communicate prompt-edit risk before downstream consumers encounter the change.

## VI. THREATS TO VALIDITY

**Construct.** SemVer for code is anchored in machine-checkable API compatibility; prompts have no analogous contract. Our  $\ell_{\text{emp}}$  is itself an operationalization. Reported numbers therefore characterize agreement between two principled but designed labelers, not absolute ground truth.

**Internal and baseline strength.** The same team designed the catalog and the rule-guided procedure. We mitigate by releasing all artifacts and comparing against a Simple-LLM baseline that sees the same pairs without the structured procedure. The baseline performs close to PROMPTVER, especially on XSum, limiting accuracy claims; we therefore frame the contribution as audibility and reproducibility.

**Empirical-label construction.** Hard-violation-aware labels encode SemVer assumptions about parser and contract failures. They are necessary because semantic-distance metrics cannot detect all breaking changes, but they make  $\ell_{\text{emp}}$  a designed operational reference rather than independent behavioral ground truth. This also creates partial overlap with PROMPTVER’s contract checks. We therefore treat agreement as evidence of consistency with an explicit release-risk model, not as proof of semantic correctness. The replication package includes both metric-only and hard-violation-aware labels.

**External and reliability.** Tasks are English-language classification and summarization; results may not transfer to extraction, tool use, RAG, multilingual settings, or agentic workflows. Only  $P$  is varied empirically;  $H$ ,  $C$ ,  $T$  are framework-level and untested. The judge (gpt-4o) is distinct from executors but is conditioned on  $M$ , so cross-executor consistency is observed but not proven.

## VII. RELATED WORK

PROMPTVER sits at three intersections. *Semantic versioning* was codified for communicating release impact [12], with empirical work documenting its value and imperfect adherence on machine-checkable APIs [3], [9], [14]. Prompts lack such a surface, motivating adaptation rather than direct application.

*Prompts as software artifacts.* Tafreshipour et al. [21] mined prompt-edit commits, Liang et al. [6] surveyed practitioner workflows and found ad-hoc version control, others proposed structured composition [8], [11], [16], and prompt-program frameworks expose typed surfaces at compile time [1], [5], [10], [13]. Closer to runtime, Shankar et al. [20] treat prompt evolution as a testing problem. Across this work, prompts are decomposed, surveyed, structured, and tested, but no thread assigns each edit a release-time severity label downstream consumers can act on.

*LLM judges and prompt sensitivity.* Prior work uses LLMs as judges [2], [7], [23] or quantifies sensitivity to semantically equivalent rephrasings [4], [19], the latter motivating our paraphrastic noise floor. PROMPTVER’s judge differs in that the artifact judged is a prompt-edit pair (not a model output) and the target is a structured three-class label tied to a versioning discipline.

## VIII. CONCLUSION

Prompt edits are becoming a routine part of software evolution, but developers lack a disciplined way to communicate change impact. PROMPTVER adapts SemVer to prompt edits through a versioning unit, template decomposition, an edit taxonomy, and a rule-guided classifier that produces an auditable decision trace. In a controlled benchmark of 436 valid edit-executor rows, it reaches 0.718 agreement with hard-violation-aware empirical labels (against a 0.670 always-PATCH floor with 1.000 dangerous-miss rate), with stronger performance on classification than summarization. A direct LLM baseline is competitive and outperforms PROMPTVER on XSum, so the main value of PROMPTVER is not raw accuracy but the reproducible decomposition of severity decisions into task,

contract, component, and rule-level checks. Qualitative analysis shows that this trace enables root-cause diagnosis even on disagreement cases. We report preliminary evidence on classification and summarization; broader transfer, generation-specific calibration, safety-diagnostic reporting, and ablation of the non- $P$  versioning-unit elements remain future work.

**AI use.** An LLM-based writing tool was used for language editing and clarity. All technical contributions, experimental design, analyses, and conclusions are the authors’ own.

## REFERENCES

- [1] L. Beurer-Kellner, M. Fischer, and M. Vechev, “Prompting Is Programming: A Query Language for Large Language Models,” *Proc. ACM PLDI*, 2023.
- [2] C.-H. Chiang and H.-Y. Lee, “Can Large Language Models Be an Alternative to Human Evaluations?” *Proc. ACL*, 2023.
- [3] A. Decan and T. Mens, “What do package dependencies tell us about semantic versioning?” *IEEE TSE*, vol. 47, no. 6, pp. 1226–1240, 2019.
- [4] O. Honovich, U. Shaham, S. R. Bowman, and O. Levy, “Instruction Induction: From Few Examples to Natural Language Task Descriptions,” *Proc. ACL*, 2023.
- [5] O. Khattab et al., “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” *Proc. ICLR*, 2024.
- [6] J. T. Liang et al., “Prompts Are Programs Too! Understanding How Developers Build Software Containing Prompts,” *Proc. ICSE*, 2025.
- [7] Y. Liu et al., “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,” *Proc. EMNLP*, 2023.
- [8] S. Morales, R. Clarisó, and J. Cabot, “A DSL for Testing LLMs for Fairness and Bias,” *Proc. MODELS*, 2024.
- [9] S. Mostafa, R. Rodríguez, and X. Wang, “Experience Paper: A Study on Behavioral Backward Incompatibilities of Java Software Libraries,” *Proc. ISSTA*, 2017.
- [10] R. Willard et al., “Outlines: Structured Text Generation,” <https://github.com/dottxt-ai/outlines>, 2024.
- [11] G. Perrone and S. P. Romano, “PEaC: Prompt Engineering as Code – An Approach for Prompts as a First-Class Citizen,” *arXiv preprint arXiv:2412.02173*, 2024.
- [12] T. Preston-Werner, “Semantic Versioning 2.0.0,” <https://semver.org/>, 2013.
- [13] Pydantic AI Contributors, “Pydantic AI: Agent Framework for Production Generative AI,” <https://ai.pydantic.dev/>, 2024.
- [14] S. Raemaekers, A. van Deursen, and J. Visser, “Semantic versioning and impact of breaking changes in the Maven repository,” *J. Syst. Softw.*, vol. 129, pp. 140–158, 2017.
- [15] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, “Beyond Accuracy: Behavioral Testing of NLP Models with CheckList,” *Proc. ACL*, 2020.
- [16] K. Ronanki, B. Cabrero-Daniel, and C. Berger, “Prompt Smells: An Omen for Undesirable Generative AI Outputs,” *Proc. ICSE Workshops*, 2024.
- [17] E. Schluntz, “Engineering Prompts: Lessons from Production LLM Applications,” Industry talk, 2024.
- [18] T. Schnabel and J. Neville, “Symbolic Prompt Program Search,” *arXiv preprint arXiv:2404.02319*, 2024.
- [19] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, “Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design,” *Proc. ICLR*, 2024.
- [20] S. Shankar et al., “SPADE: Synthesizing Data Quality Assertions for LLM Pipelines,” *Proc. VLDB*, 2024.
- [21] A. Tafreshipour et al., “Prompting in the Wild: An Empirical Study of Prompt Evolution in Software Repositories,” *Proc. ICSE*, 2025.

- [22] J. Wang *et al.*, “Software Testing With Large Language Models: Survey, Landscape, and Vision,” *IEEE TSE*, 2024.
- [23] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Proc. NeurIPS Datasets & Benchmarks*, 2023.